appendix B Neo4j

Throughout the book, examples, code, and exercises are based on a specific graph database: Neo4j. Nevertheless, all the theories, the algorithms, and even the code can be easily adapted to work with any graph database. We selected this database because

- We know it inside and out.
- It is a native graph database (with all the related consequences, as explained in the book).
- It has a broad community of experts.

According to DB-Engines (**https://db-engines.com/en/ranking_trend/graph+dbms**), Neo4j has been the most popular graph DBMS for a number of years (figure B.1).

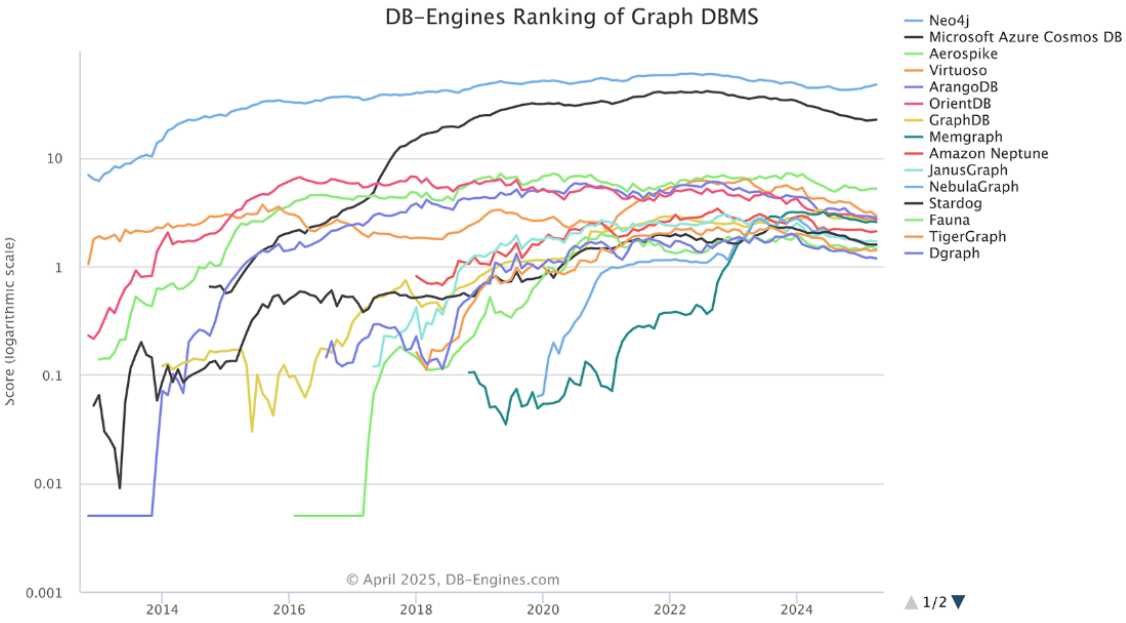

Figure B.1 DB-Engines ranking of graph DBMSs

NOTE DB-Engines scoring considers multiple factors, from the number of mentions on websites to the frequency of technical questions on Stack Overflow and DBA Stack Exchange, and from the number of related jobs offered to its relevance in social network.

This appendix provides the minimum amount of information necessary to get started with Neo4j and use it in the book. We introduce Neo4j, give installation instructions, and describe the Cypher language (the language used to query the database). We'll also discuss the configuration of some plugins used in the examples.

B.1 Introduction to Neo4j

Neo4j is available as a GPL3-licensed, open source Community Edition. Neo4j Inc. also licenses an Enterprise Edition with backup, scaling extensions, and other enterprise-grade features under closed source commercial terms. Neo4j is implemented in Java and is accessible over the network through a transactional HTTP endpoint or through the binary Bolt protocol (**https://boltprotocol.org/**). It's widely adopted due to the following features:

- It implements a labeled property graph database.
- It uses native graph storage based on index-free adjacency. (For a discussion of graph representations, see appendix A.)
- It provides native graph querying and a related language, Cypher (**www.opencypher.org**), which defines how the graph database describes, plans, optimizes, and executes queries.
- Every architecture layer—from queries using Cypher to files on disk—is optimized for storing and retrieving graph data.
- It provides an easy-to-use developer workbench with a graph visualization interface.

Neo4j provides a full-strength, industrial-grade database, and transactional support is one of its many strengths. This differentiates it from many NoSQL solutions. It provides full ACID support [2], defined as follows:

- *(A) Atomicity*—You can wrap multiple database operations within a single transaction and make sure they're all executed atomically. If one of the operations fails, the entire transaction is rolled back.
- *(C) Consistency*—When you write data to the Neo4j database, you can be sure that every client accessing the database will read the latest data.
- *(I) Isolation*—Operations in a single transaction are isolated from one another so that writes in one transaction won't affect reads in another transaction.

- *(D) Durability*—Neo4j writes your data to disk, and it becomes available after a database restart or a server crash.

This support makes it easy for anyone used to traditional relational database guarantees to transition to Neo4j and makes working with graph data both safe and convenient.

In addition to ACID transactional support, other features to consider when choosing the right database for an architectural stack include the following:

- *Recoverability*—This has to do with the database's ability to set things right after a failure. Databases, like any other software system,

  > *… are susceptible to bugs in their implementation, in the hardware they run on, and in that hardware's power, cooling, and connectivity. Though diligent engineers try to minimize the possibility of failure in all of these, at some point it's inevitable that a database will crash. And when a failed server resumes operation, it must not serve corrupt data to its users, irrespective of the nature or timing of the crash. When recovering from an unclean shutdown, perhaps caused by a fault or even an overzealous operator, Neo4j checks in the most recently active transaction log and replays any transactions it finds against the store. It's possible that some of those transactions may have already been applied to the store, but because replaying is an idempotent action, the net result is the same: after recovery, the store will be consistent with all transactions successfully committed prior to the failure [1].*

Moreover, Neo4j offers an online backup procedure that lets you recover the database when the original data is lost. In such a case, recovery to the last committed transaction is impossible, but it is better than losing all the data [1].

- *Availability*—To increase the chance of recoverability,

  > *A good database needs to be highly available to meet the increasingly sophisticated needs of data-heavy applications. The database's ability to recognize and, if necessary, repair an instance after crashing means that data quickly becomes available again without human intervention. And of course, more live instances increases the overall availability of the database*

to process queries. It's uncommon to want individual disconnected database instances in a typical production scenario. More often, we cluster database instances for high availability. Neo4j uses a master/slave cluster arrangement to ensure that a complete replica of the graph is stored on each machine. Writes are replicated out from the master to the slaves at frequent intervals. At any point, the master and some slaves will have a completely up-to-date copy of the graph, while other slaves will be catching up (typically, they will be but milliseconds behind) [1].

- *Capacity*—Related to the amount of data it is possible to store in a database or, in our specific case, in a graph database, the adoption of dynamically sized pointers in Neo4j 3.0 and higher allows the database to scale up to run any size of graph workload with an upper limit "in the quadrillions" of nodes [3].

Two excellent books on this topic are *Graph Databases* [1] and *Neo4j: The Definitive Guide* [2]. At the time of writing, the latest Neo4j version available was *2025.x*, so the code and the queries for this book were tested using this version.

B.2 Installing Neo4j

As mentioned, Neo4j is available in two editions: Community and Enterprise. You can download the Community Edition freely from the Neo4j website and use it indefinitely for noncommercial purposes, respecting the GPLv3 license (**https://www.gnu.org/licenses/gpl-3.0.en.html**). You can download the Enterprise Edition and try it for a limited time under specific constraints (it requires you to buy a proper license). The book's code works perfectly with the Community Edition, so we recommend using it. That way, you have time to evaluate Neo4j. Alternatively, you can use Neo4j packaged as a Docker image.

Another option is to use the Neo4j Desktop (v2) GUI (**https://neo4j.com/docs/desktop/current/**). Neo4j Desktop is a sort of developer environment for Neo4j. You can manage as many projects and database servers locally as you like, and you can also connect to remote Neo4j servers. Neo4j Desktop comes with a free developer's license for Neo4j Enterprise Edition. From the Neo4j download page (**https://neo4j.-**

com/deployment-center/), you can select which one of the editions you would like to download and install.

B.2.1 Installing a Neo4j server

If you decide to download a Neo4j server (either Community or Enterprise), the installation is straightforward. On Linux or Mac, follow these steps:

1. Make sure you have Java 21 (or later) installed.
2. Open your terminal/shell.
3. Extract the contents of the archive using `tar xf <filecode>` (for example, `tar xf neo4j-community-2025.08.0-unix.tar.gz`).
4. Place the extracted files in a permanent home on your server. The top-level directory is referred to as NEO4J_HOME.
5. To run Neo4j
    1. As a console application, use `<NEO4J_HOME>/bin/neo4j console`.
    2. As a background process, use `<NEO4J_HOME>/bin/neo4j start`.
6. Visit http://localhost:7474 in your web browser.
7. Connect using the username *neo4j* with the default password *neo4j*. You will then be prompted to change the password.

On Windows machines, the procedure is similar. Unzip the downloaded file, and proceed with the previous steps. At the end of the process, when you open the specified link in the browser, you'll see something like figure B.2.

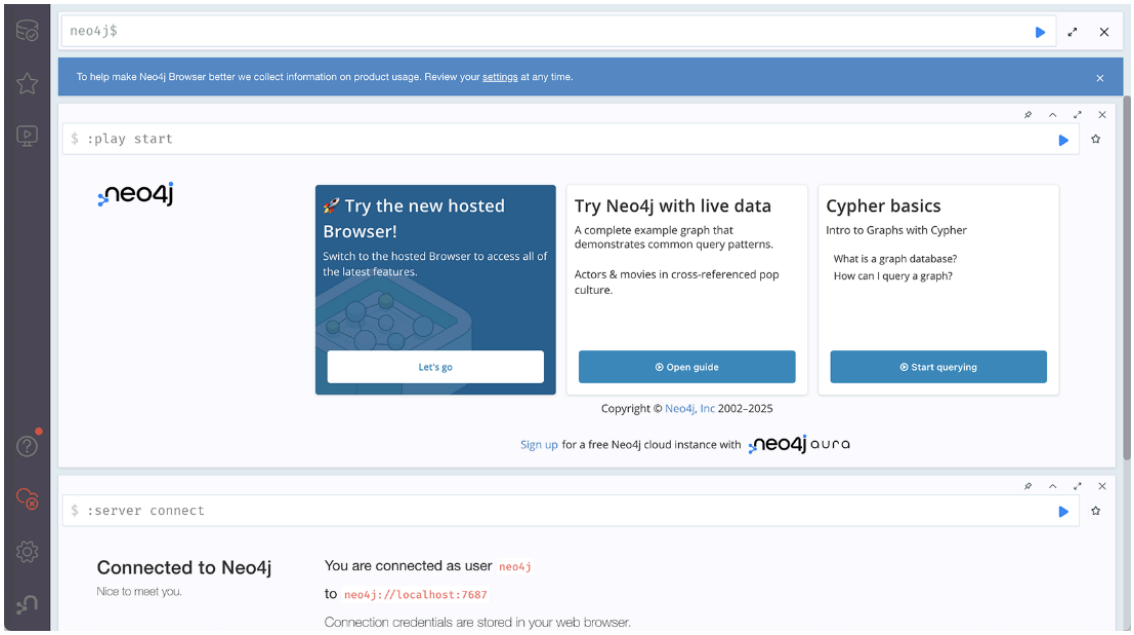

Figure B.2 An image of the Neo4j browser

The Neo4j browser, a simple web-based application, allows users to interact with a Neo4j instance, submit queries and perform basic configurations. At this point, you should be ready to go.

B.2.2 Neo4j Desktop installation

When you download Desktop, the installation procedure is available in the installation guide. If you need to, refer to that guide for your specific machine's operating system. To get Desktop up and running quickly, the directions for macOS are as follows:

1. In your Downloads folder, locate and double-click the .dmg file. This starts the Neo4j Desktop installer.
2. Save the app to the Applications folder (either the global one or your user-specific one) by dragging and dropping the Neo4j Desktop icon to the folder (figure B.3).

Figure B.3 Saving the Neo4j Desktop app on macOS

31. Locate the Neo4j icon in the Applications folder, and double-click it to launch Desktop (figure B.4).

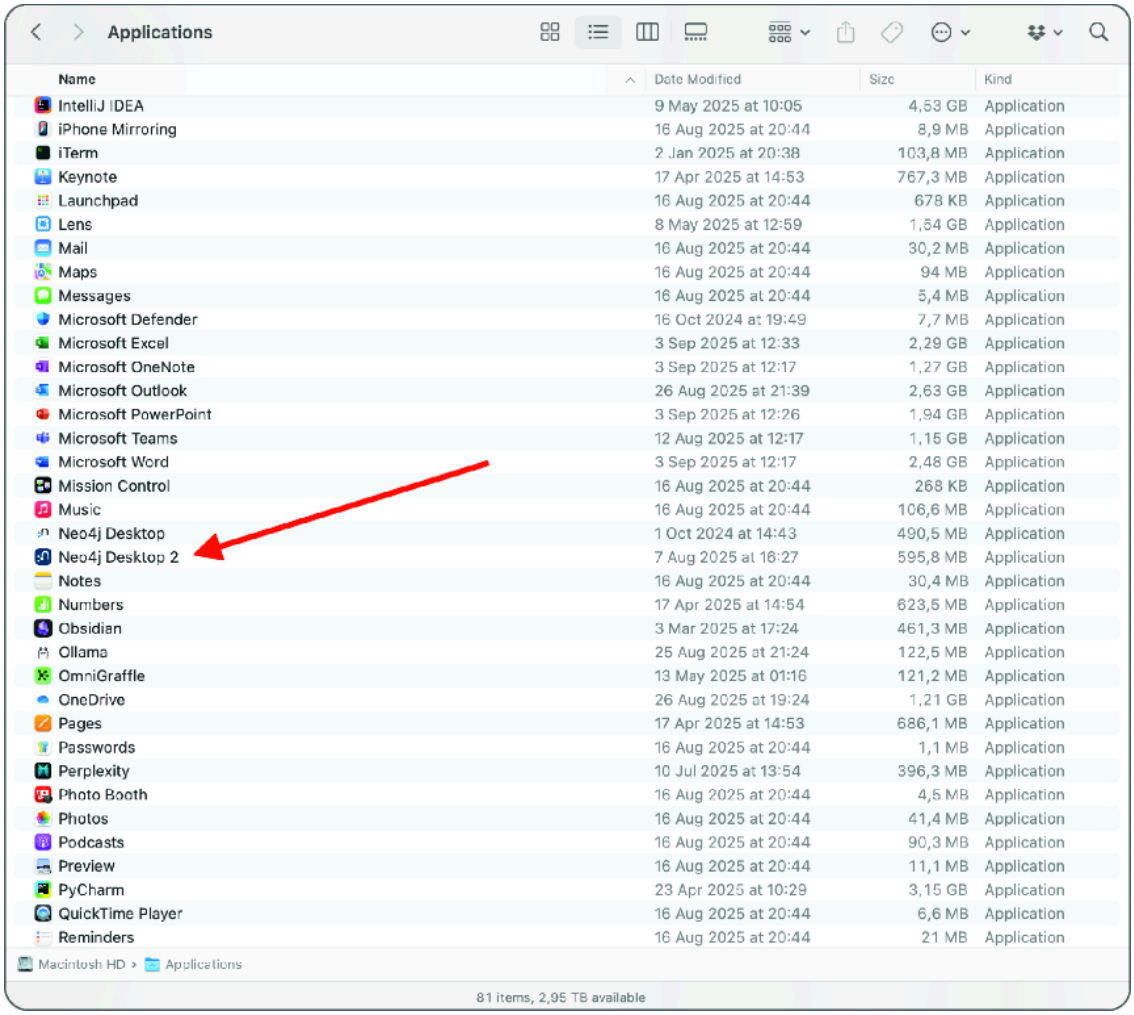

Figure B.4 Launching Neo4j Desktop

41. Once you've activated Desktop, create your first instance by clicking Create instance (figure B.5).

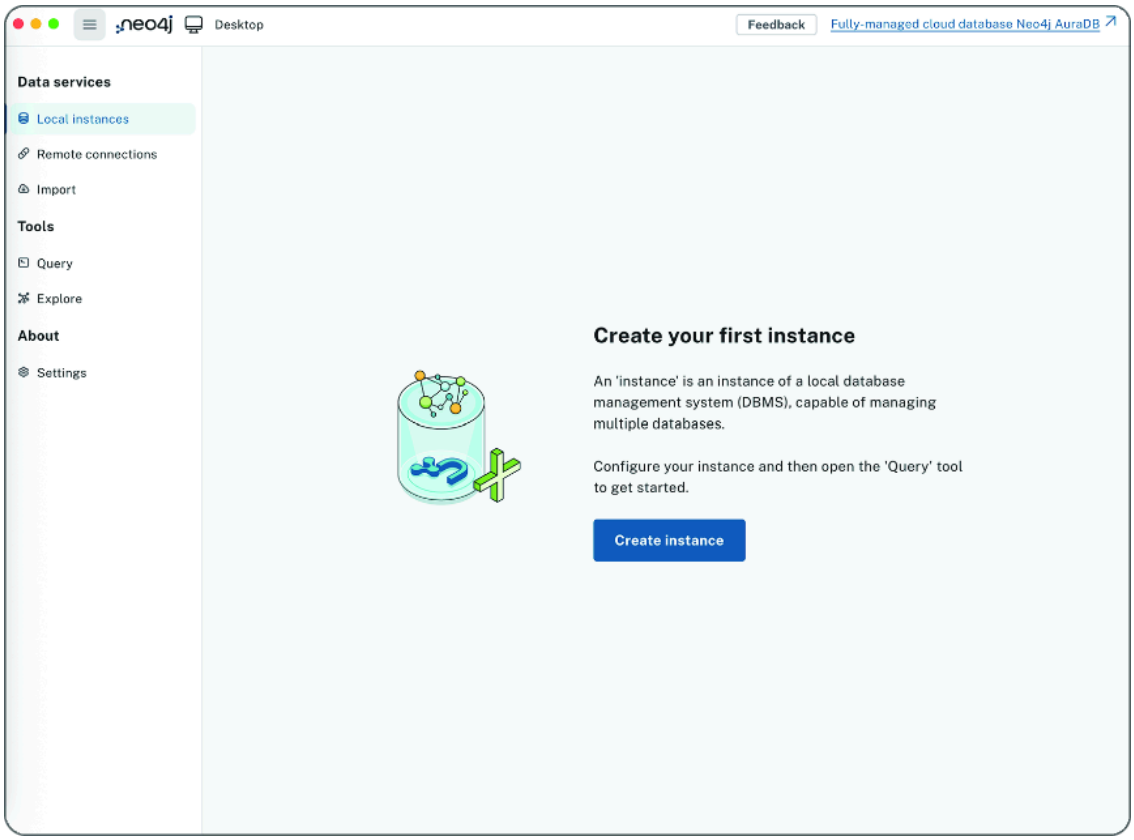

Figure B.5 Creating the first local instance in Neo4j Desktop

51. Specify the Instance name, the Neo4j version, and the Password for the neo4j user (figure B.6).

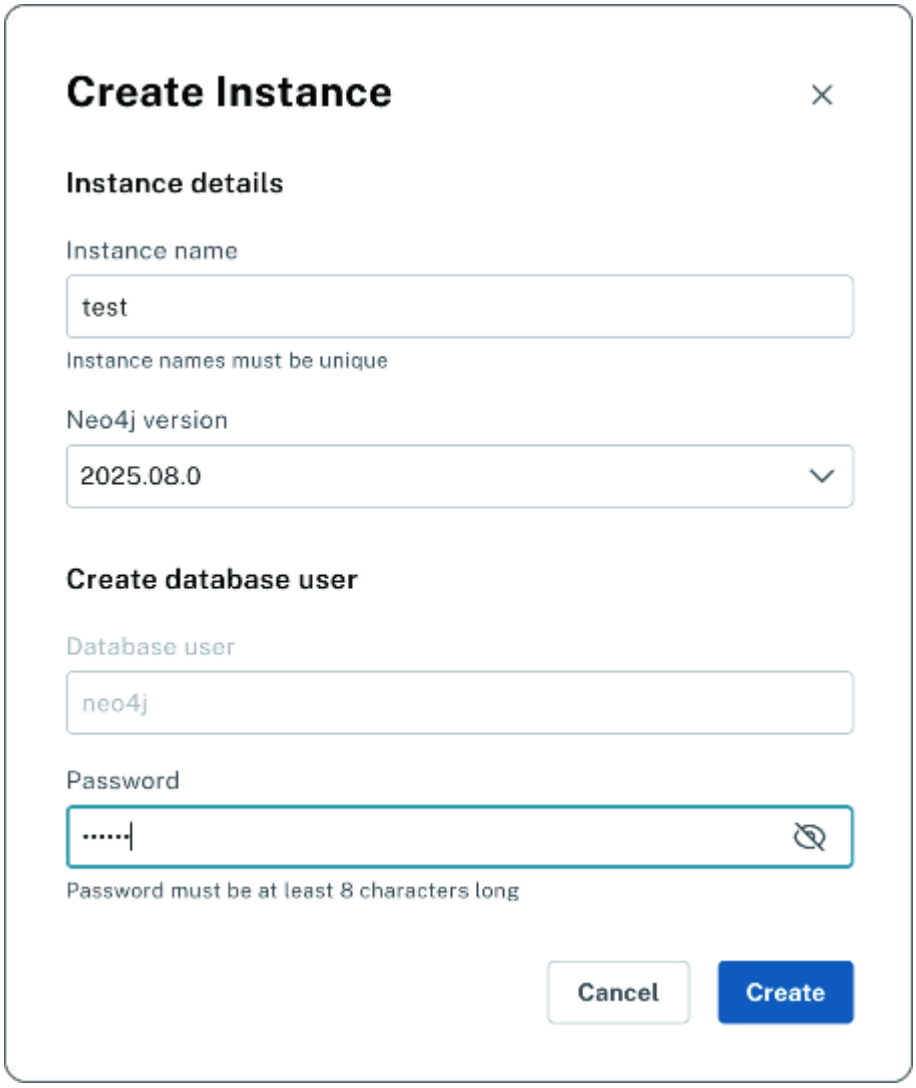

Figure B.6 Adding and selecting a new local graph or connecting to an existing one

61. At this point you can create a database in the instance. A default one, called `neo4j`, is created together with the `system` database (figure B.7).

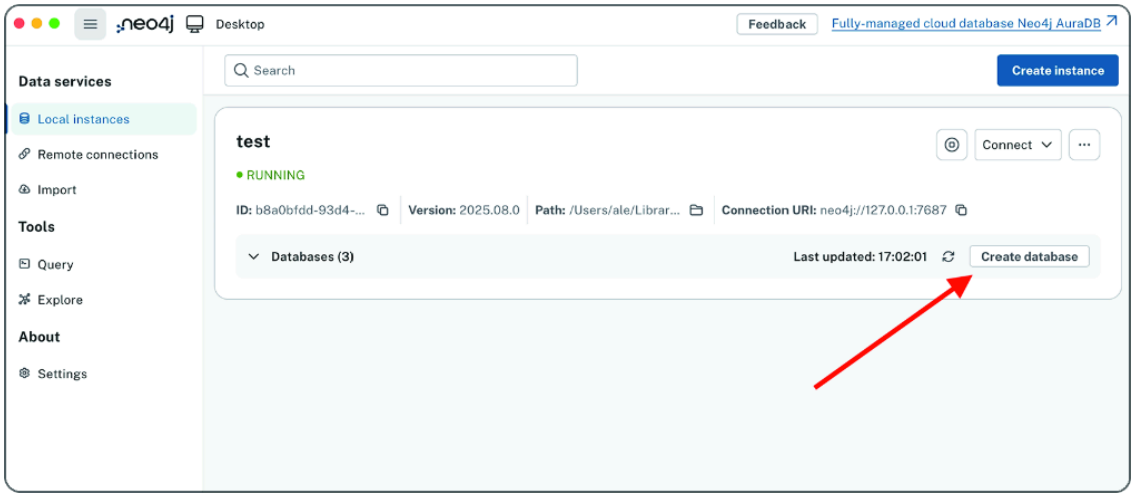

Figure B.7 Creating a database in the local instance

7. You can start and stop the instance using the button close to the instance name (figure B.8).

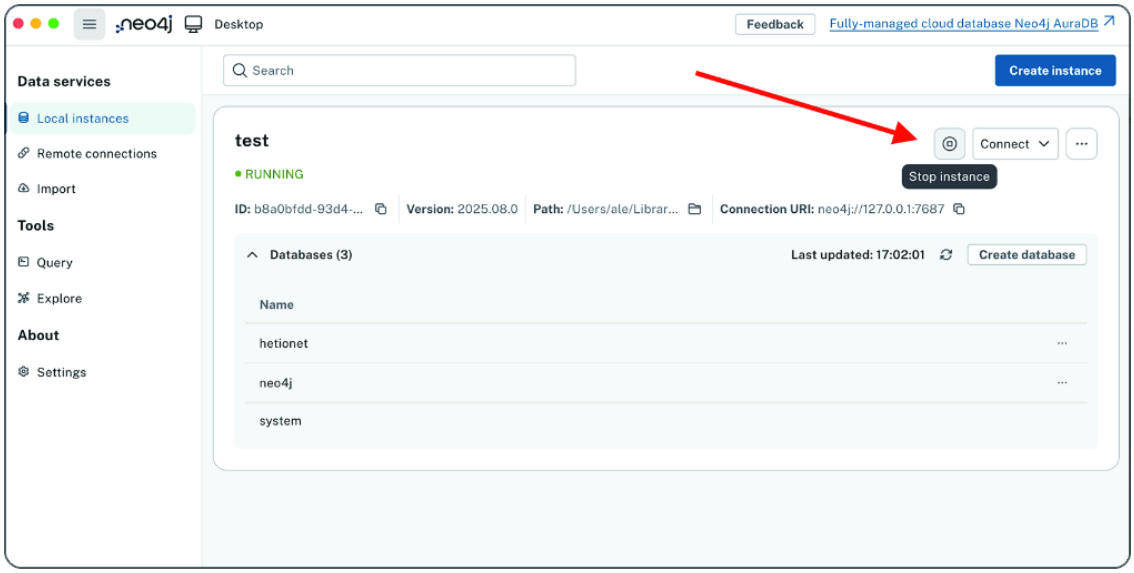

Figure B.8 Starting the new database instance

8. Click the Connect button to use the instance via the Neo4j browser (Query) or the exploration interface (Explore) (figure B.9).

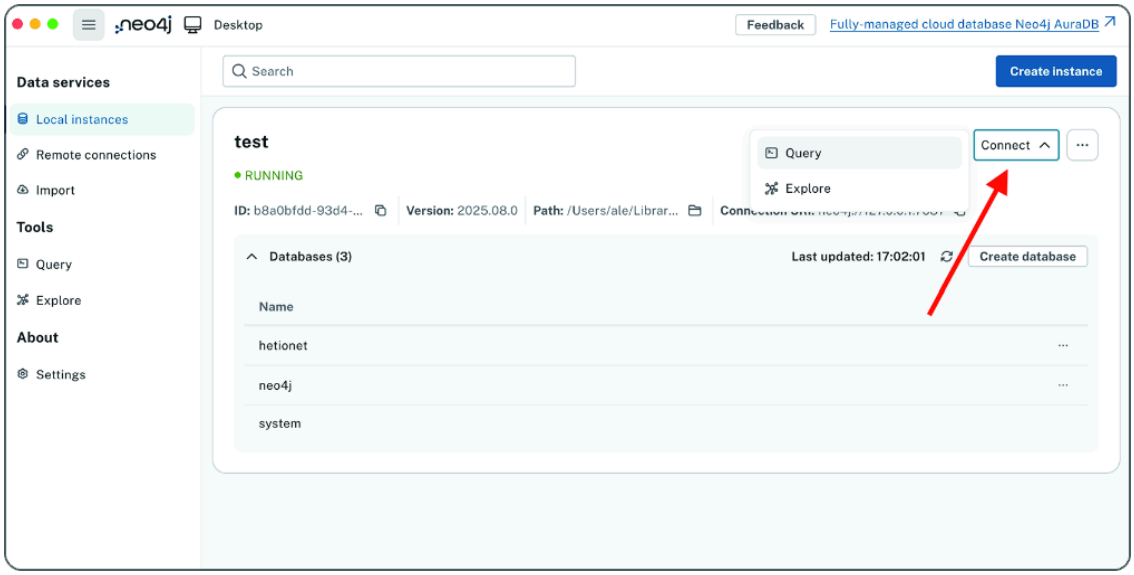

Figure B.9 Opening the Neo4j Browser

The result will be the same as in the steps leading to figure B.2. You will have access to the browser, where you can interact with Neo4j.

If you would like to avoid all this effort, Neo4j also has a cloud version called Aura (**https://neo4j.com/product/auradb/**). At the time of writing, a free tier version is available if you want to play around a bit before jumping into the examples and exercises in the book. Keep in mind that for the exercises, in terms of the learning curve, it would be better to have Neo4j installed locally on your machine or where you can run the Python code.

B.3 Cypher

Neo4j uses Cypher (**https://neo4j.com/developer/cypher/**) for its query language. Like SQL (which inspired it), Cypher enables users to store and retrieve data from a graph database. Via Cypher, Neo4j provides a language that is easy to learn, understand, and use, while also incorporating the power and functionality of other standard data-access languages.

Cypher is a declarative language for describing visual patterns in graphs using ASCII Art syntax. Using this syntax, it is possible to describe a graph pattern visually and logically. The following is a simple example where we are looking for all the nodes of type `Person` in the graph.

Listing B.1 The simplest query

```
MATCH (p:Person)
RETURN p
```

We can use this pattern to search for nodes and relationships in the graph or to create them. We can state what we want to select, insert, update, or delete from our graph data without describing exactly how to do that.

Cypher is open source. The openCypher (**www.opencypher.org**) project provides an open language specification, technical compatibility kit, and reference implementation of the parser, planner, and runtime for Cypher. It is backed by several companies in the database industry and allows implementors of databases and clients to freely benefit from, use, and contribute to the development of the openCypher language.

Throughout the book, we provide examples and exercises to help you learn this language. If you would like to read more about Cypher, I recommend Neo4j's guide (**https://neo4j.com/developer/cypher/**) as a good reference, as it's chock-full of examples.

B.4 Installing plugins

A remarkable thing about Neo4j is how easily you can extend it. Developers can customize it in many ways, and you can enrich the Cypher language with new procedures and functions you can call when

querying the graph. You can customize its security with authentication and authorization plugins. Moreover, you can enable new surfaces to be created in the HTTP API via server extensions.
You'll find a lot of preexisting plugins to download, configure, and use. The most relevant are developed by Neo4j and supported by the entire community (because they are open source). For the purposes of the book and the examples presented, we use two of them:

- *The Awesome Procedures on Cypher library (APOC)*—A standard utility library containing common procedures and functions. It is the most widely used extension library for Neo4j. It provides functionality for utilities, conversions, graph updates, and more.

This library is well-supported and extremely easy to run as a separate function or to include in Cypher queries. This allows developers across platforms and industries to use a standard library for common procedures and only write their own functionality for business logic and specific needs.

- *The Graph Data Science library (GDS)*—The analytics engine of Neo4j, which makes it possible to address complex questions about system dynamics and group behavior. This library of procedures exploits the predictive power of relationships and network structures in existing data to answer previously intractable questions, increasing prediction accuracy. Data scientists benefit from a customized, flexible data structure for global computations and a repository of powerful, robust algorithms to quickly compute results over tens of billions of nodes.

The following subsections describe how to download, install, and configure these libraries. We recommend following the directions in these sections before reading chapter 3 where we start using Cypher queries.

B.4.1 Installing APOC Core

Installing plugins in Neo4j is simple. Let's start with the APOC library (**https://github.com/neo4j/apoc/releases**).

If you installed the server version, download the plugin from the related GitHub release page (select the version that matches your version of Neo4j: 2025.07.*x*, 2025.08.*x*, and so forth). Copy it to the plugins directory

in your NEO4J_HOME folder. Now, edit the configuration file conf/neo4j.-conf, adjusting or adding the following lines to that file:

```
dbms.security.procedures.unrestricted=apoc.*
dbms.security.procedures.allowlist=apoc.*
```

Restart Neo4j and open the browser. Run the following procedure to check whether everything is in place.

Listing B.2 Checking whether APOC is correctly installed

```
CALL dbms.procedures() YIELD name
WHERE name STARTS WITH "apoc"
RETURN name
```

You should see a list of APOC procedures.

If you are using the desktop version, this process is even simpler. After creating the database (see section B.2.2 up to step 6), open the instance, click the three dots at upper right, and select Plugins (figure B.10). Select the plugins you would like to install, and then restart the instance.

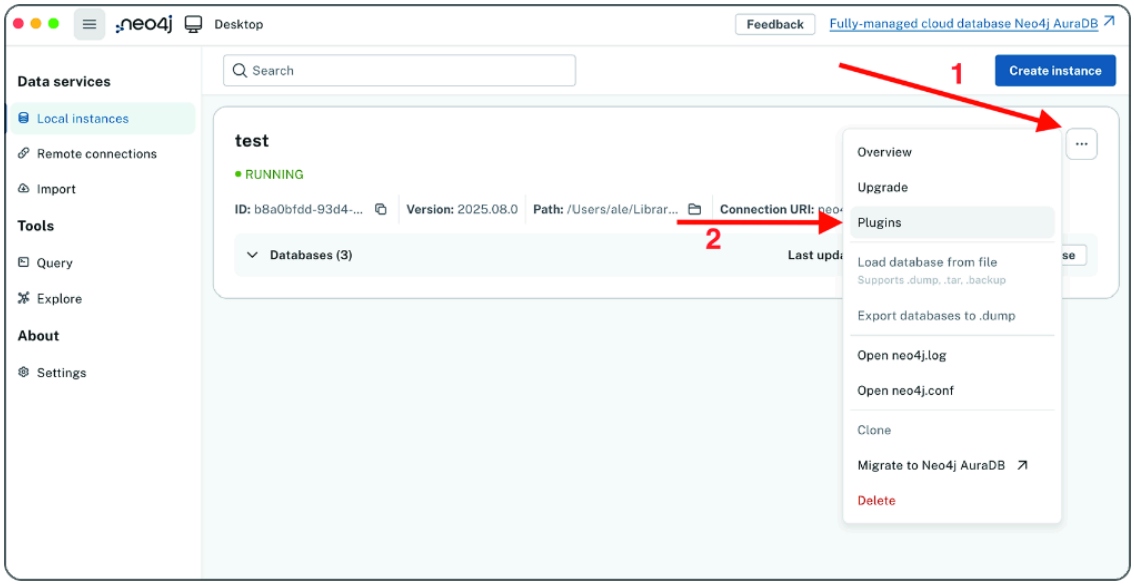

Figure B.10 APOC installation from the Neo4j Desktop

For further details and explanation, see the official APOC installation guide. You'll find it at **https://neo4j.com/labs/apoc/**.

B.4.2 GDS installation

You can follow a similar procedure to install the GDS library. If you installed the Neo4j server version, download the plugin from the related GitHub release page (**https://github.com/neo4j/graph-data-science/releases**). Copy the *-standalone.jar file to the plugins directory in your NEO4J_HOME folder. Now edit the configuration file conf/neo4j.conf, adjusting or adding the following lines:

```
dbms.security.procedures.unrestricted=apoc.*,gds.*
dbms.security.procedures.allowlist=apoc.*,gds.*
```

Restart Neo4j and open the browser. Run the following procedure to check that everything is in place.

Listing B.3 Checking whether GDS is correctly installed

```
RETURN gds.version()
```

You should see the version of the GDS you downloaded.

If you use Neo4j Desktop, then after creating the database (see section B.2.2 up to step 6), follow the same procedure as for APOC but select the Graph Data Science plugin (figure B.11).

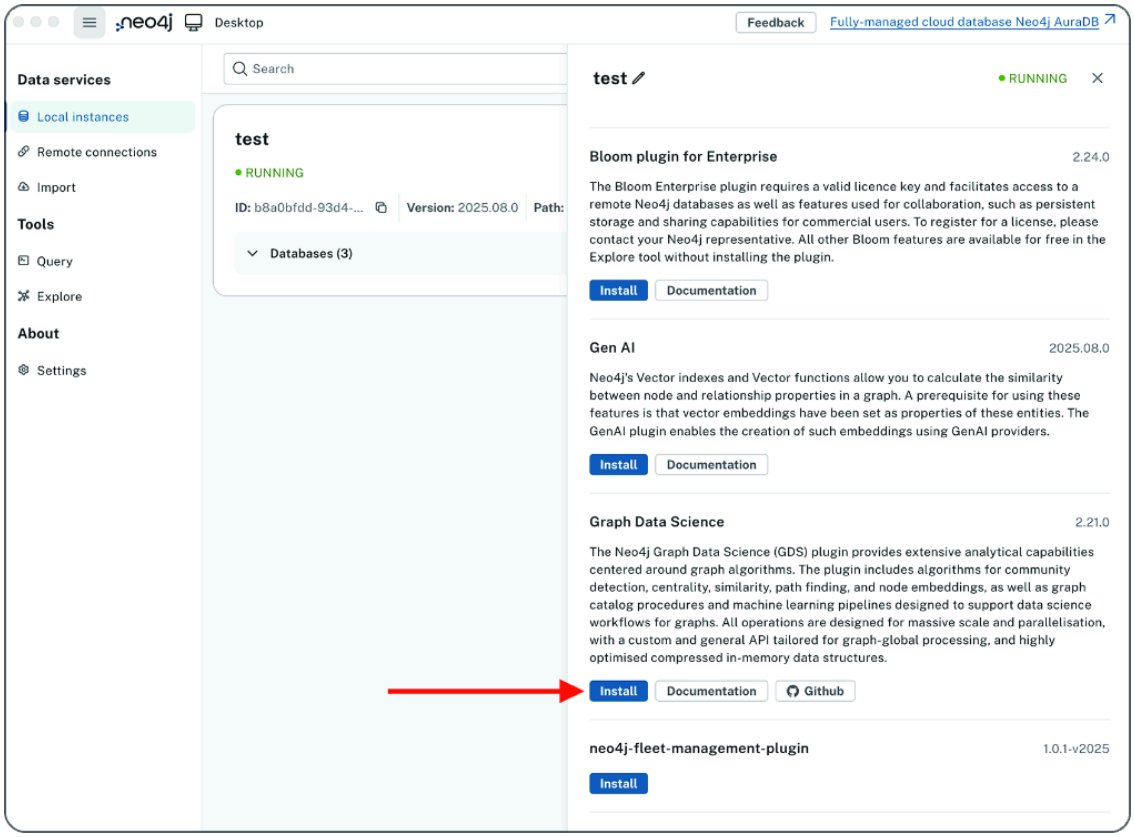

Figure B.11 GDS installation from the Neo4j Desktop

After these steps, you are ready to have fun with Neo4j. Be sure to run all the examples and exercises in the book.

B.5 Cleaning

Sometimes you may need to clean up your database. You can do it using the functions available in the APOC library that you just installed into your database. The following two listings provide the code.

Listing B.4 Deleting everything

```
CALL apoc.periodic.iterate('MATCH (n) RETURN n',
    'DETACH DELETE n', {batchSize:1000})
```

Listing B.5 Dropping all constraints

```
CALL apoc.schema.assert({}, {})
```